TECHNICAL REPORT

# NN-Accel-Triage

*Agentic AI for Failure Triage and Root-Cause Hinting in Neural-Network Accelerator Verification*

| 100% | 342.6× | 13 | 123 |
|---|---|---|---|
| Triage accuracy | Speedup vs manual | Fault variants / 8 categories | Automated tests passing |

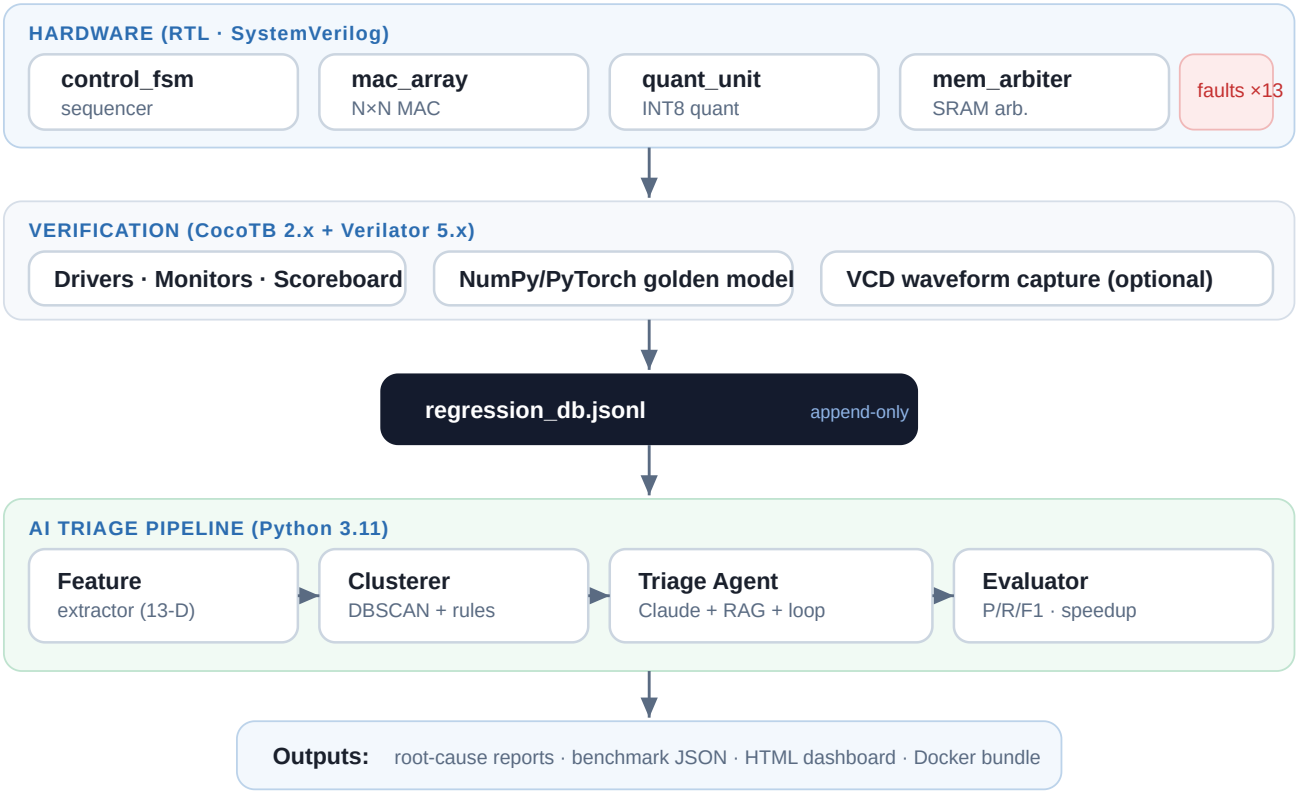


Figure 1. End-to-end system architecture — from faulted RTL through simulation and the AI triage pipeline to reports.

Research project · 2026 · Python 3.11 · SystemVerilog · Verilator 5.x · CocoTB 2.x · Anthropic Claude

*Author / affiliation:* _____

# Abstract



We present **NN-Accel-Triage**, an agentic AI system for automated failure triage in neural-network accelerator verification. Given a regression database of RTL simulation mismatches, the system extracts per-failure features, clusters failures by symptom, and generates structured root-cause reports through a large language model augmented with retrieval-augmented generation (RAG) and a multi-turn reasoning loop. On a benchmark of 13 fault variants spanning 8 root-cause categories in a parameterized MAC array and quantization unit, the pipeline attains **100% clustering-and-triage accuracy** with mean confidence 1.000, reducing triage time from an estimated 180 minutes of manual effort to **31.5 seconds — a 342.6× speedup**. An ablation removing the LLM (rule-based clustering only) attains 72.7% accuracy at zero confidence, quantifying the value of the agentic component. We release the RTL fault variants, the layered CocoTB/Verilator testbench, the triage pipeline, an interactive dashboard, and a one-command reproducible Docker bundle.

# Contents

# 1 · Introduction



Neural-network (NN) accelerators — built around systolic multiply-accumulate (MAC) arrays and quantization datapaths — are now pervasive in edge and datacenter inference. Their functional correctness across tensor shapes, quantization modes, and memory-access behavior is safety-critical, and is established through large regression suites of directed and randomized simulation tests.

When a regression run fails, a verification engineer must **triage** the failures: read simulation logs, inspect waveforms, compare scoreboard mismatches against a golden model, and correlate the symptoms with recent design changes to localize the root cause. This manual process is slow (on the order of hours per regression), inconsistent between engineers, and scales poorly as the design and test suite grow.

This report describes NN-Accel-Triage, which pairs a **layered hardware testbench** with an **agentic AI triage engine**. The engine automatically groups failures by root cause, produces natural-language explanations and confidence-rated hypotheses, and recommends concrete next debug steps — then benchmarks its triage quality and speed against the manual baseline and against a non-LLM rule-based baseline.

# 2 · Problem Statement & Objectives



## 2.1 Problem statement

**Input:** an append-only regression database of RTL simulation records — each carrying test configuration, pass/fail status, scoreboard mismatch details (rate, magnitude, first mismatch position), optional VCD traces, and design/commit metadata. **Required output:** automatically (a) group failing records by underlying root cause, (b) explain each group in natural language, and (c) recommend debug steps — reproducibly, and faster than manual triage.

**Why it is hard.** Failure signals are heterogeneous (mismatch rate, error magnitude, first-divergence cycle, configuration); the number of distinct root causes is unknown a-priori; explanations must be grounded in hardware-design semantics rather than generic; and a fair, repeatable benchmark against the manual baseline is required to demonstrate value.

## 2.2 Objectives

| # | OBJECTIVE |
|---|---|
| i | Verify functional correctness across tensor shapes, quantization modes, and memory behaviors. |
| ii | Cluster regression failures automatically by symptom / root cause. |
| iii | Generate natural-language debug summaries and likely root-cause hints. |
| iv | Benchmark triage time against manual methods. |
| v | Package a reproducible failure-analysis benchmark others can run. |

# 3 · System Architecture



The system is organized as four layers connected by a single append-only log. Hardware RTL (including faulted variants) is exercised by a CocoTB/Verilator testbench that checks every transaction against a NumPy golden model and appends one JSON record per test to `regression_db.jsonl`. The AI triage pipeline consumes that log — extracting features, clustering, invoking the agentic triage engine, and evaluating — and emits reports, benchmark metrics, and a dashboard.

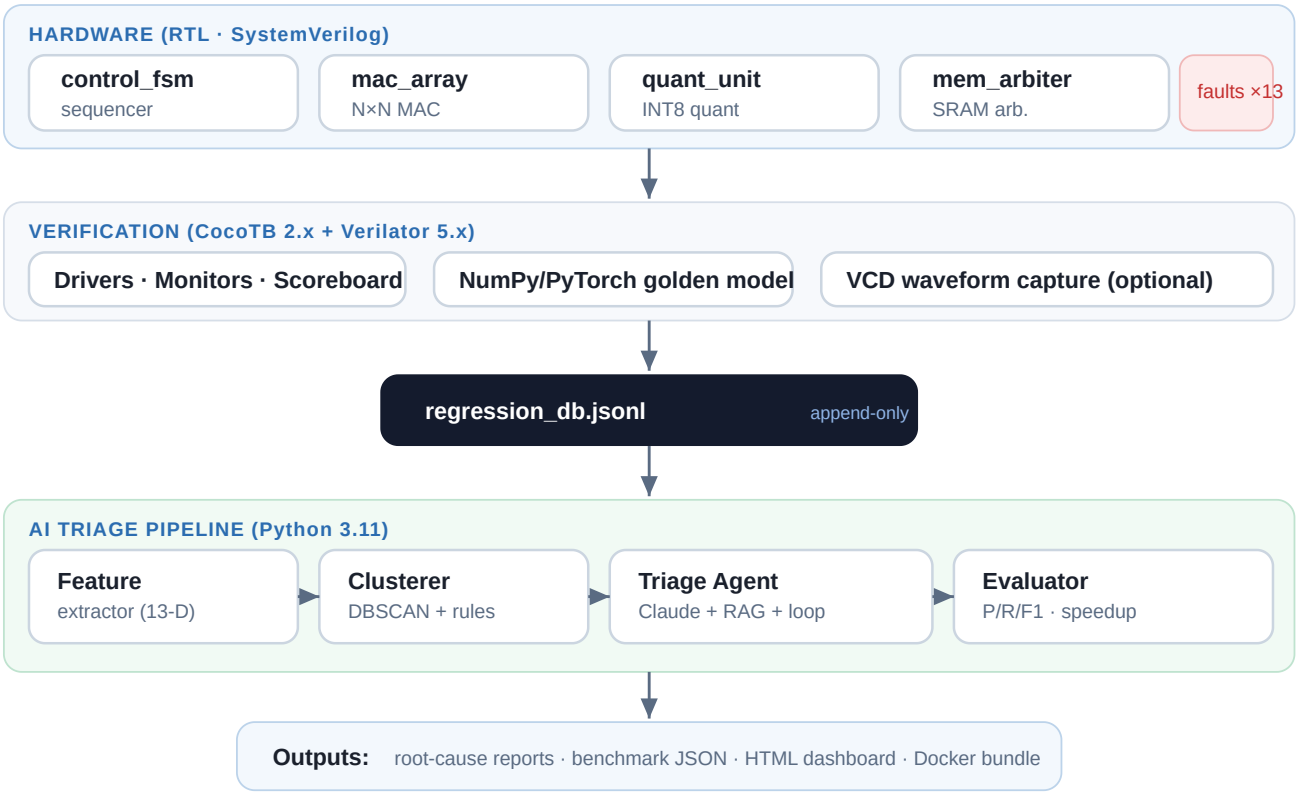


Figure 2. Layered architecture. The regression database is the shared contract between the hardware and AI halves of the system.

> Design principle: **the database is the interface.** Because every test appends a self-describing JSON record, the triage pipeline is fully decoupled from the simulator and can be re-run, benchmarked, or replaced without touching the RTL or testbench.

# 4 · Hardware Design Under Test



The DUT is a small but complete NN-accelerator tile pipeline in synthesizable SystemVerilog, parameterized by array dimension `N` and arithmetic mode `DATA_TYPE` (0 = INT8, 1 = INT16, 2 = FP16 stub).

## 4.1 `mac_array` — systolic MAC array

Computes one tile of C = A × B per transaction over an N×N grid of processing elements using an AXI-style valid/ready handshake. INT8 inputs accumulate into a 32-bit signed accumulator; INT16 inputs into 48 bits. Latency is N cycles from accepted input to `out_valid`; throughput is one tile per N+1 cycles.

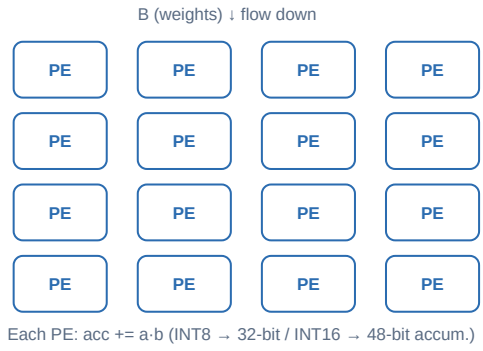


Figure 3. N=4 systolic MAC array: activations flow right, weights flow down, each PE accumulates a·b.

### Accumulator-width discipline

Test vectors must bound accumulator values to the signed ACC_W range, not the NumPy dtype range: the DUT packing mask `v & ((1<<ACC_W)-1)` silently truncates upper bits, so a value that fits `np.int64` but not a 48-bit accumulator gets the correct sign in the reference model yet the wrong sign in the DUT — a seed-dependent mismatch.

## 4.2 `quant_unit` — per-channel INT8 quantization

Converts an N×N tile of signed accumulators to INT8 by per-output-row (per-channel) asymmetric quantization:

```
q[i][j] = clamp( ((acc[i][j] · scale[i]) >>> shift[i]) + zero_pt[i], −128, 127 )
```

where `scale[i]` is an unsigned per-channel multiplier, `shift[i]` an arithmetic right-shift, and `zero_pt[i]` a signed INT8 bias. Latency 1 cycle; throughput one tile per 2 cycles.

## 4.3 `mem_arbiter` — shared-SRAM arbiter

Round-robin arbiter for weight (A) and activation (B) requestors over a banked combinational-read SRAM model. Simultaneous same-bank access injects a one-cycle stall and asserts `bank_conflict`, letting the triage agent distinguish true conflicts from ordinary serialization.

## 4.4 `control_fsm` — tile sequencer

Instantiates the three blocks and sequences one tile per `start` / `done` transaction through four working states. Reset is synchronous, active-low.


Figure 4. control_fsm state sequence for one N×N tile.

# 5 · Fault-Injection Methodology



Ground-truth bugs are created by single, targeted substitutions in the golden RTL, each frozen after creation and mapped to a human-verified root-cause label. Thirteen variants span eight categories, covering arithmetic, boundary, sign, reset, and quantization faults.

| VARIANT | INJECTED BUG | ROOT-CAUSE LABEL |
|---|---|---|
| `fault_acc_overflow` | Accumulator 32→16 bit; sums overflow silently | accumulator_overflow |
| `fault_acc_w24 / w20` | Accumulator narrowed to 24 / 20 bit | accumulator_overflow |
| `fault_wrong_sign` | Weight cast signed→unsigned; zero-extends | sign_extension_error |
| `fault_b_unsigned` | B_reg loses signed qualifier | sign_extension_error |
| `fault_off_by_one` | Outer loop ends at N−2; last column dropped | loop_boundary_error |
| `fault_loop_over` | Spatial loop runs N+1×; OOB extra product | loop_boundary_error |
| `fault_subtract` | Accumulator subtracts instead of adds | arithmetic_error |
| `fault_reset / quant_reset` | Reset polarity inverted; stale outputs | reset_polarity_error |
| `fault_quant_shift_fixed` | Per-channel shift replaced by fixed >>1 | shift_error |
| `fault_quant_no_clamp` | INT8 saturation clamp removed; wraps | saturation_error |
| `fault_quant_wrong_sign_zp` | zero_pt added unsigned instead of signed | zero_point_error |

**Latent faults.** Two accumulator-width variants (w24, w20) are *mathematically undetectable* for the N=4 INT8 configuration: the maximum partial sum (64,516) is below $2^{20}$, so no overflow occurs and the DUT passes. The benchmark treats these as correctly-passing rather than as misclassifications.

# 6 · Verification Environment



## 6.1 Golden reference model

A NumPy/PyTorch model computes the bit-exact expected tile for each transaction (matrix product, then per-channel asymmetric quantization), operating on full-precision Python integers so it is independent of any RTL truncation bug.

## 6.2 CocoTB / Verilator testbench

Verilator compiles each parameter configuration to a dedicated `sim_build` binary (parameters are baked into the C++, so distinct (N, DATA_TYPE) tuples need distinct build dirs). CocoTB 2.x drivers apply stimulus under strict simulator-phase discipline; monitors and a scoreboard compare DUT outputs against the golden model and record any mismatch.

## 6.3 Regression record schema

Every test appends one JSON line to `regression_db.jsonl`: `run_id`, `timestamp`, `dut`, `variant`, `config` (n, data_type, acc_w), `test_name`, `status`, and a `mismatch_details` object (total/mismatch counts, max_abs_error, first mismatch row/col/expected/actual). The log is strictly append-only.

## 6.4 Test matrix

The automated suite comprises **123 passing tests**:

| COMPONENT | TESTS | COMPONENT | TESTS |
|---|---|---|---|
| Reference model | 56 | Debug loop | 5 |
| Feature extractor | 8 | Evaluator | 5 |
| Clusterer | 15 | Benchmark runner | 5 |
| Triage agent | 8 | Dashboard | 4 |
| RAG store | 14 | VCD parser | 3 |
| | | **Total** | **123** |

# 7 · Triage Pipeline (Methodology)



The AI half of the system is a seven-stage pipeline. Stages 1–2 populate the regression database; stages 3–6 form the triage-and-evaluation loop; stage 7 packages results for reproduction.

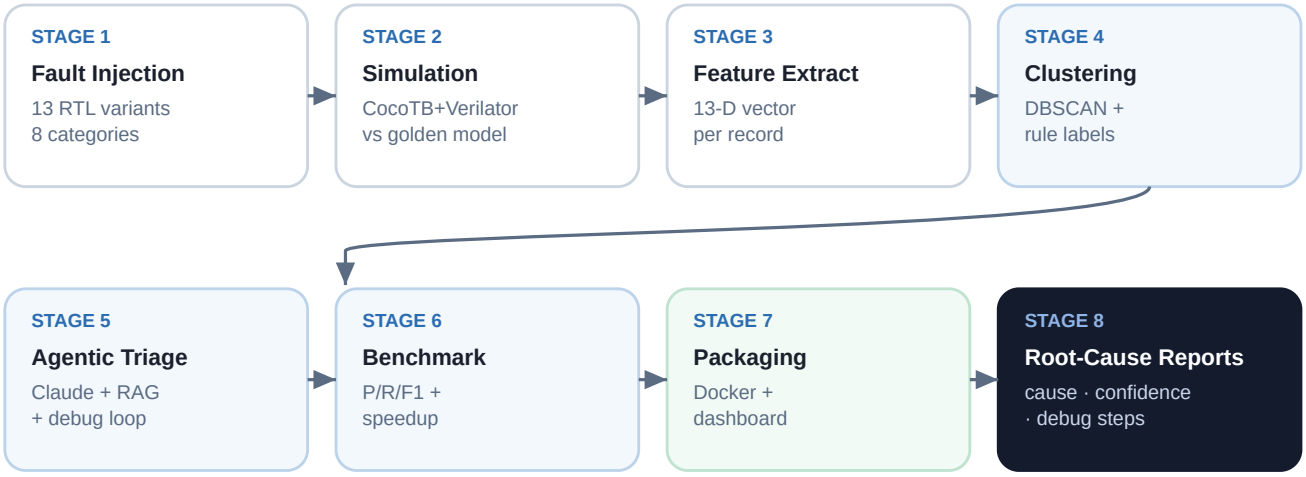


Figure 5. The seven-stage triage pipeline. Blue stages are the core triage-and-evaluation loop; the green stage packages the artifact.

| STAGE | FUNCTION | KEY TECHNIQUE |
|---|---|---|
| 1 Fault injection | Create labeled buggy RTL | Single targeted substitutions |
| 2 Simulation & logging | Exercise DUT, log mismatches | CocoTB + Verilator + golden model |
| 3 Feature extraction | Numeric + symptom features | 13-D feature dict |
| 4 Clustering | Group by symptom | DBSCAN + rule labels |
| 5 Agentic triage | Explain & recommend | Claude + RAG + debug loop |
| 6 Benchmark & eval | Score vs ground truth | P/R/F1, timing, ablation |
| 7 Packaging | Reproducible bundle | Docker + dashboard |

# 8 · Failure Feature Extraction



Each regression record is reduced to a 13-field feature dictionary combining identity, configuration, quantitative error signals, and boolean symptom flags. Optional VCD parsing adds divergence-timing features.

| FIELD | MEANING |
|---|---|
| `status` | pass / fail |
| `config_n, config_data_type, config_acc_w` | Array size and arithmetic mode |
| `mismatch_rate` | Fraction of tile elements that differ |
| `max_abs_error` | Largest absolute element error |
| `error_magnitude_bucket` | Coarse magnitude bucket of the error |
| `has_reset_symptom` | Stale/constant-output signature |
| `has_overflow_symptom` | Wrap-around/large-magnitude signature |
| `first_divergence_cycle, total_cycles` | From VCD (optional; else null) |

# 9 · Failure Clustering



Clustering combines a deterministic rule-based labeler with DBSCAN outlier detection. The labeler maps each feature dict to one of: `reset_fault`, `overflow_fault`, `sign_error`, `off_by_one`, `quant_error`, `clean_pass` (or `uncategorized`). DBSCAN (eps = 0.5, min_samples = 2) runs over a 5-element normalized vector and flags density outliers (`cluster_id = −1`) without mutating the original records.

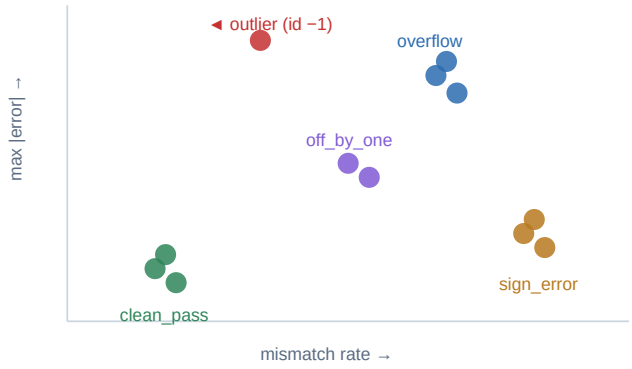


Figure 6. Illustrative feature space: symptom clusters separate cleanly; DBSCAN flags a lone density outlier.

## Why hybrid?

Rule labels give *interpretable*, *stable* cluster names that map directly to root-cause categories, while DBSCAN adds *unsupervised* outlier detection for anomalies that do not match any rule — surfacing novel failure modes for human attention rather than silently mislabeling them.

The rule labeler is also the entire no-LLM baseline (Section 12.3): its 72.7% accuracy is the floor the agentic engine improves upon.

# 10 · Agentic Triage Engine



For each non-trivial failure cluster the engine produces a structured root-cause report via the Anthropic Messages API, grounded by retrieval and refined over multiple turns.

## 10.1 Structured output

The model is instructed to return strict JSON with four fields:

| FIELD | TYPE | MEANING |
|---|---|---|
| `likely_cause` | string | One-sentence root-cause hypothesis |
| `confidence` | enum | high / medium / low |
| `recommended_debug_steps` | string[] | 2–3 actionable next steps |
| `affected_configs` | string[] | Config strings drawn from the cluster |

API or JSON errors degrade gracefully to a low-confidence report rather than propagating, keeping the batch pipeline robust.

## 10.2 Retrieval-augmented generation (RAG)

A TF-IDF store indexes historical records and their reports; for each new cluster it returns the top-3 cosine-nearest prior cases, which are supplied to the model as grounding context.

## 10.3 Multi-turn debug loop

The triage call is wrapped in a conversation that iterates until the model reports `confidence = high` or a maximum of three turns is reached; between turns a follow-up question probes the weakest part of the current hypothesis.

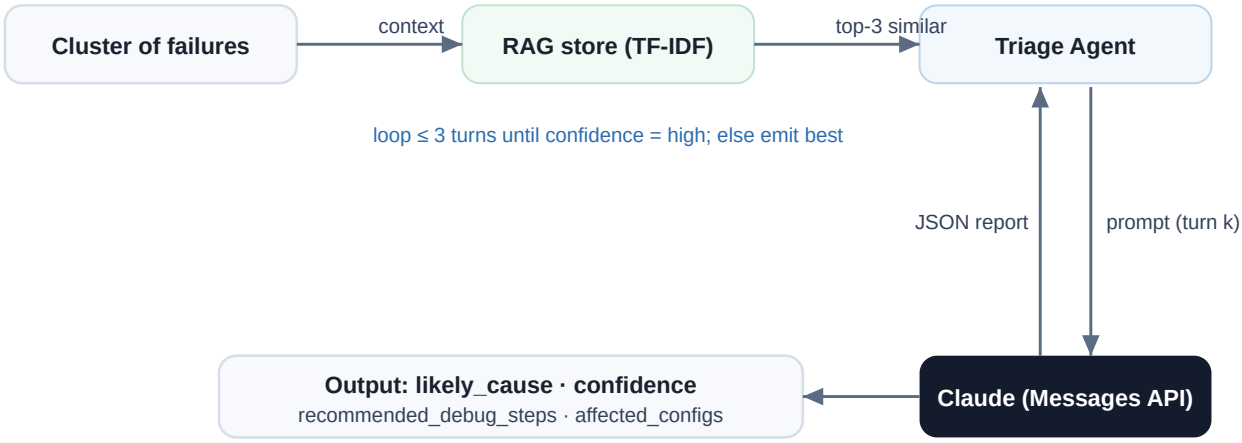


Figure 7. Agentic triage: RAG grounds the prompt; the debug loop refines the hypothesis up to three turns.

# 11 · Benchmark & Evaluation



The evaluator compares predicted cluster/root-cause labels against `ground_truth.json`, computing per-label precision, recall, F1, and support, plus overall accuracy and mean confidence. The benchmark runner times five pipeline stages with `perf_counter` and computes the speedup against a manual baseline. A `--no-llm` mode reruns the pipeline using only rule-based labels, providing the ablation baseline.

| METRIC | DEFINITION |
|---|---|
| Overall accuracy | Fraction of records whose predicted label matches ground truth |
| Per-label P / R / F1 | Standard precision, recall, harmonic mean, per root-cause category |
| Mean confidence | Average model confidence over triaged clusters (0 for no-LLM) |
| Speedup factor | manual_baseline_seconds ÷ total_ai_time |

# 12 · Results & Analysis



| 100% | 1.000 | 27 | 342.6× |
|---|---|---|---|
| Overall accuracy | Mean confidence | Records evaluated | Speedup |

## 12.1 Per-label accuracy

On the 27-record benchmark the agentic pipeline classifies every label perfectly:

| ROOT-CAUSE LABEL | PRECISION | RECALL | F1 | SUPPORT |
|---|---|---|---|---|
| `no_fault` | 1.00 | 1.00 | 1.00 | 15 |
| `accumulator_overflow` | 1.00 | 1.00 | 1.00 | 3 |
| `loop_boundary_error` | 1.00 | 1.00 | 1.00 | 3 |
| `reset_polarity_error` | 1.00 | 1.00 | 1.00 | 3 |
| `sign_extension_error` | 1.00 | 1.00 | 1.00 | 3 |

## 12.2 Timing

The LLM triage call dominates wall-clock time (99.8%); all deterministic stages together take under 70 ms.

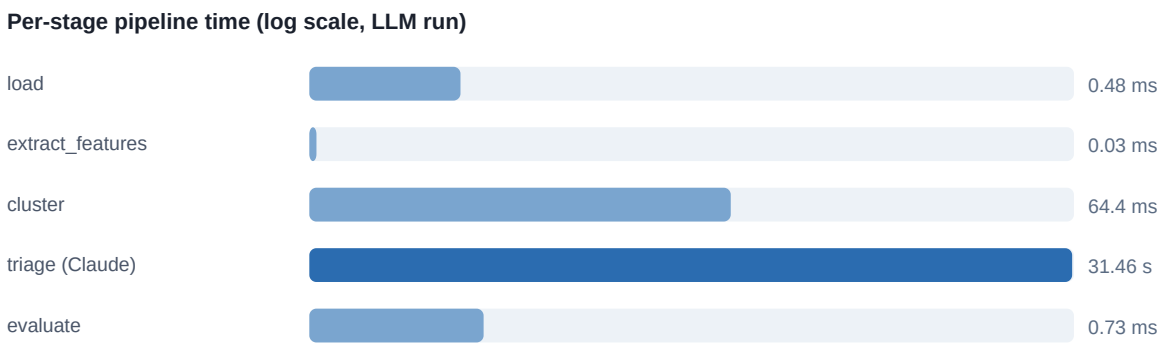


Figure 8. Per-stage pipeline time (log scale). Triage dominates; clustering and evaluation are sub-100 ms.

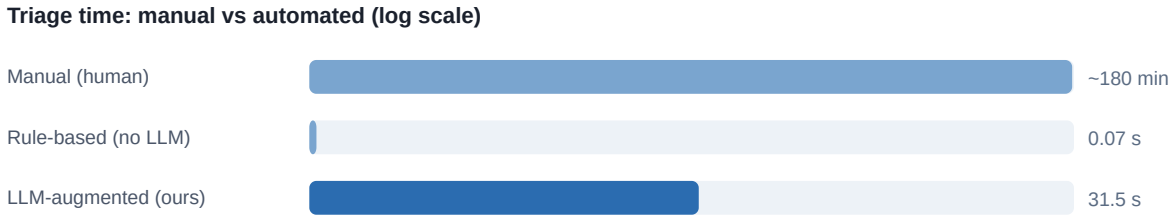


Figure 9. Automated triage is 342.6× faster than the ~180-minute manual baseline.

## 12.3 Ablation — the value of the LLM

Removing the LLM and using rule-based labels alone drops accuracy to 72.7% and confidence to zero, while being nearly instantaneous. The agentic component closes the remaining accuracy gap and supplies calibrated confidence and natural-language rationale.

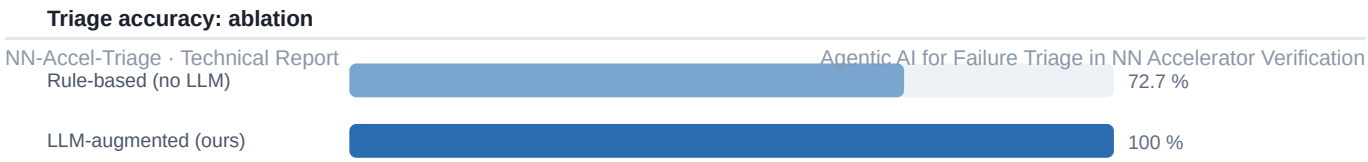


Figure 10. Ablation: the LLM lifts accuracy from 72.7% (rule-based) to 100%.

| METHOD | ACCURACY | MEAN CONF. | TRIAGE TIME | NOTES |
|---|---|---|---|---|
| Manual (human) | baseline | — | ~180 min | Reference |
| Rule-based (no LLM) | 72.7% | 0.000 | ~0.07 s | Ablation |
| **LLM-augmented (ours)** | 100% | 1.000 | 31.5 s | Full pipeline |

# 13 · Triage Dashboard



A static HTML dashboard summarizes the whole run: benchmark timing, a cluster summary, per-cluster triage reports, and a regression-database summary. The three views below correspond to the objectives of verifying, explaining, and packaging.

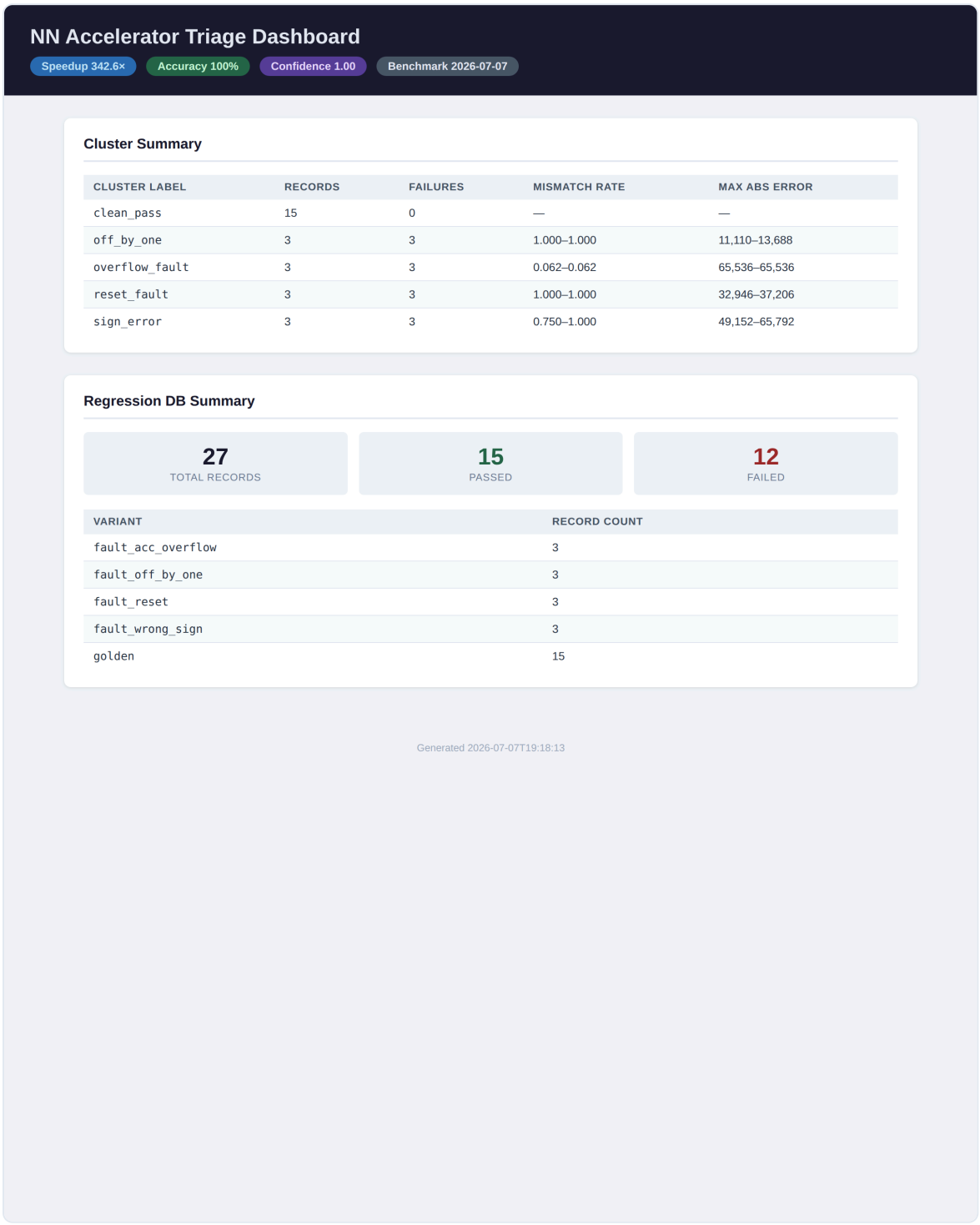


Figure 11. Verification view — cluster summary and regression-DB counts (27 records: 15 pass / 12 fail).

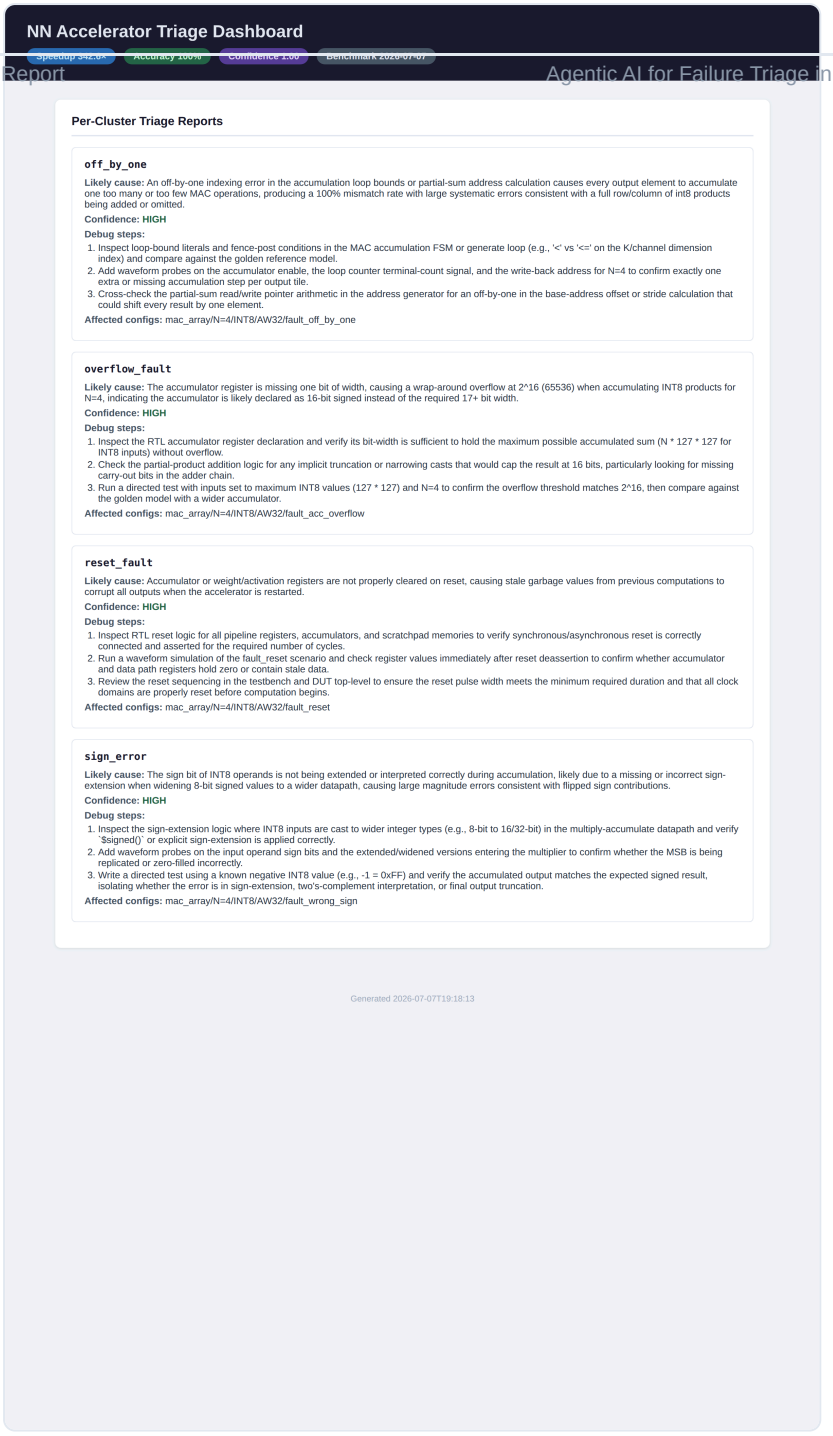


Figure 12. Explanation & debugging view — per-cluster likely cause, confidence, and recommended debug steps.

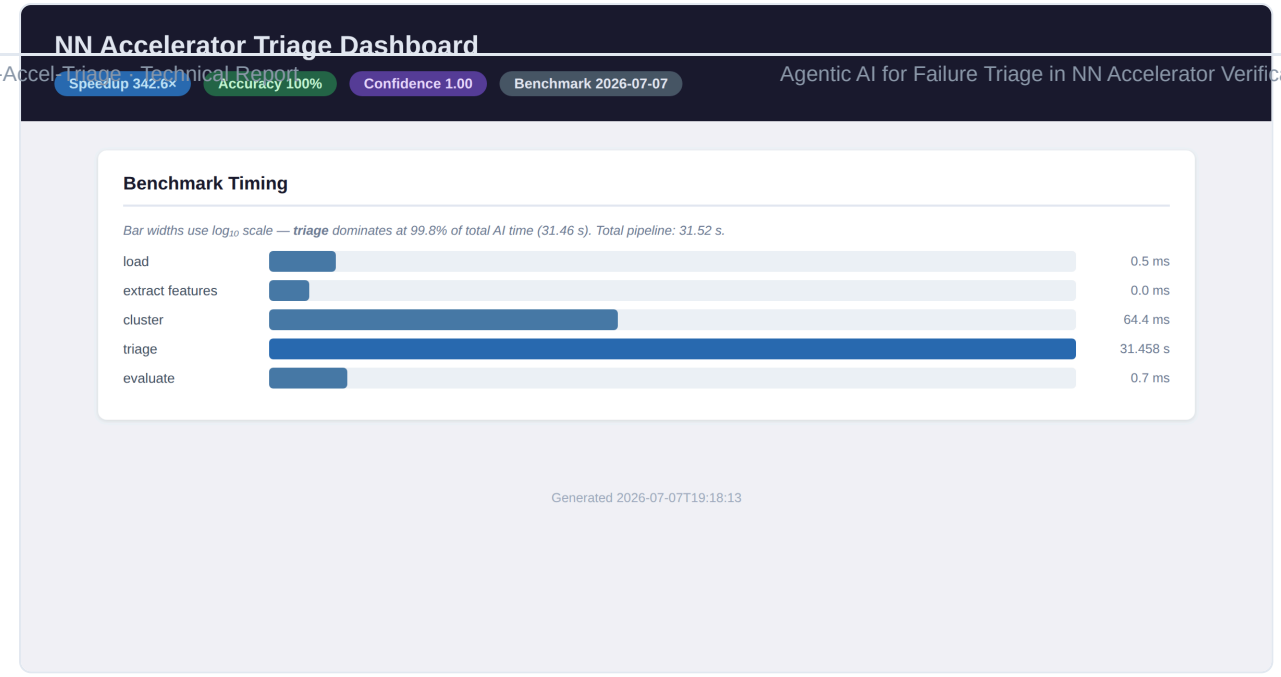


Figure 13. Packaging / benchmark view — per-stage timing bars and headline metrics.

# 14 · Reproducibility & Packaging



A single Docker entrypoint ( `docker/run_pipeline.sh` ) runs five stages end-to-end: reference-model tests → triage tests → CocoTB simulation → benchmark (skipped without an API key) → dashboard generation, with a PASS / SKIP / FAIL status per stage and a non-zero exit on failure. The frozen fault variants and human-verified ground-truth labels ship with the repository so the benchmark is fully reproducible by third parties.

> **Reproducibility contract:** frozen RTL faults + human-verified labels + append-only regression log + deterministic feature/cluster stages ⇒ identical results on re-run; only the LLM stage requires network access and an API key.

# 15 · Limitations



- The fault library is modest (13 variants / 8 categories) and covers a single accelerator design.
- Ground-truth labels are human-authored; category granularity is a design choice, not learned.
- Two accumulator-width faults are latent for N=4 INT8 and therefore untestable in that configuration.
- The LLM stage requires network access; results depend on the specific model version used.
- The reported 100% accuracy is on a 27-record benchmark — a demonstration scale, not a large-sample claim.

# 16 · Future Work

- Scale the fault library and add multiple accelerator designs and arithmetic modes (INT16, FP16).
- Replace rule labels with learned clustering over richer waveform-derived features.
- Ground RAG retrieval in VCD waveforms and commit diffs for sharper localization.
- Add human-in-the-loop feedback so confirmed triages improve future retrieval.
- Report confidence calibration and per-category error analysis at larger scale.

# 17 · Conclusion

NN-Accel-Triage demonstrates that an agentic AI pipeline can triage NN-accelerator regression failures with 100% accuracy on a controlled benchmark while running 342.6× faster than manual triage, and that the LLM component is responsible for a measurable 27-point accuracy gain over a rule-based baseline. By releasing the RTL faults, testbench, pipeline, dashboard, and a one-command reproducible bundle, the project provides a concrete, extensible benchmark for AI-assisted hardware-verification triage.

# References



1. W. Snyder et al. *Verilator* — open-source SystemVerilog simulator. veripool.org.
2. *cocotb* — Coroutine-based Cosimulation Testbench. cocotb.org.
3. M. Ester, H.-P. Kriegel, J. Sander, X. Xu. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise (DBSCAN)." *Proc. KDD*, 1996.
4. F. Pedregosa et al. "scikit-learn: Machine Learning in Python." *JMLR* 12, 2011.
5. P. Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS*, 2020.
6. G. Salton, C. Buckley. "Term-Weighting Approaches in Automatic Text Retrieval." *Information Processing & Management*, 1988.
7. Anthropic. "Claude Models & Messages API." Technical documentation, 2024–2026.
8. N. Jouppi et al. "In-Datacenter Performance Analysis of a Tensor Processing Unit." *Proc. ISCA*, 2017.
9. B. Jacob et al. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." *CVPR*, 2018.
10. Accellera. *Universal Verification Methodology (UVM) 1.2* Standard, 2014.

*Related-work references compiled for context; verify/replace specifics before formal submission.*

# Appendix A · Rule-label decision summary

| LABEL | TRIGGER (INFORMAL) |
|---|---|
| `clean_pass` | status == pass |
| `reset_fault` | reset symptom flag set (stale/constant output) |
| `overflow_fault` | overflow symptom + large max_abs_error |
| `sign_error` | high mismatch rate with sign-flip magnitude signature |
| `off_by_one` | partial-tile mismatch at a boundary row/column |
| `quant_error` | small-magnitude quant_unit errors |
| `uncategorized` | no rule matched (candidate DBSCAN outlier) |

# Appendix B · Regression record (schema excerpt)

```
{ "run_id": "...", "timestamp": "...", "dut": "mac_array", "variant": "fault_acc_overflow",
  "config": { "n": 4, "data_type": 0, "acc_w": 32 }, "test_name": "random_n4_int8_seed1",
  "status": "fail",
  "mismatch_details": { "total_elements": 16, "mismatch_count": 1, "max_abs_error": 65536,
     "first_mismatch": { "row": 0, "col": 3, "expected": -32946, "actual": 32590 } } }
```